



SE507 - SOFTWARE TESTING AND EVALUATION

A Comprehensive Testing Strategy for a Library Management System

Team Members:

Tishko Salah (Test Manager)
Sara Zuhair (Code specifier)
Yassin Hussein (Code reviewer)
Ravyar Barzan (Document inspector)

Module Leader: **Dr. Hossein Hassani**

June 15, 2025

University of Kurdistan Hewlêr
Department of Computer Science and Engineering
Erbil, Kurdistan Region, Iraq

Contents

1	Introduction	4
1.1	Problem Domain and Project Rationale	4
1.2	Purpose and Contribution of This Report	5
1.3	Scope of the Testing Mandate	5
1.4	System Architecture Overview	6

List of Figures

1.1 High-Level System Architecture of the LMS.	6
--	---

List of Tables

Introduction

This report is submitted in fulfillment of the group coursework requirements for the SE507 Software Testing and Evaluation module. The primary objective of this coursework is to provide a practical, hands-on application of the fundamental principles and processes of software testing and evaluation. In accordance with the coursework guidelines, which permit the use of a ready-made application, this project undertakes a comprehensive testing and quality assurance initiative on a pre-existing software artifact: a full-stack Library Management System (LMS) sourced from the open-source repository MERNRAD on GitHub.

The work documented herein demonstrates a systematic approach to the entire testing life-cycle as applied to an existing codebase. By selecting a representative MERN-stack application, this project serves as a realistic case study in verifying software quality, ensuring functional correctness, and validating adherence to implicit and explicit requirements. The ultimate goal is to present an empirically grounded evaluation of the software's readiness for deployment, thereby bridging the theoretical concepts of software testing with their practical implementation in a real-world scenario.

1.1 Problem Domain and Project Rationale

The impetus for selecting this particular project stems from the universal problem domain it addresses: the operational inefficiencies inherent in manual or legacy library systems. The management of core library functions—such as cataloging, circulation, and user administration—presents significant logistical challenges that are amenable to technological solutions. As delineated in the system's features and our derived Software Requirements Specification (SRS), the LMS aims to provide a centralized, scalable, and efficient digital platform. It is designed to serve two distinct user cohorts: Librarians, who function as system administrators with privileged access, and Members, who are the primary consumers of the library's services.

The system's core functional domains, which form the basis of this testing investigation, include:

- **User and Access Control Management:** Implementation of secure authentication mechanisms and a role-based access control (RBAC) model to govern system access for Librarians and Members.
- **Entity Lifecycle Management:** Provision of comprehensive CRUD (Create, Read, Update, Delete) capabilities for foundational library entities, namely books, authors, and genres.
- **Circulation Management:** Automation of the complete borrowal lifecycle, from the issuance of materials to their return, including the monitoring of due dates and the maintenance of historical transaction records.

- **Interactive User Feedback System:** A feature enabling members to contribute qualitative and quantitative feedback through book reviews and ratings, fostering community engagement.

1.2 Purpose and Contribution of This Report

This document constitutes the definitive testing and evaluation report for the selected Library Management System. Its central purpose is to methodically detail the testing strategies, execution processes, and empirical outcomes derived from our quality assurance phase. The report endeavors to furnish verifiable evidence that the LMS satisfies its specified functional and non-functional requirements, thereby establishing a high degree of confidence in its deployment readiness.

The specific objectives of this report are to:

- Systematically document the entire testing lifecycle, from strategic planning and test case architecting through to execution, data logging, and results analysis.
- Present a transparent and detailed account of all executed test protocols, including quantified outcomes, identified anomalies, and defect classifications.
- Conduct a multi-dimensional evaluation of the software's quality attributes, encompassing functionality, performance, security, and usability.
- Furnish a conclusive, evidence-based assessment for stakeholders regarding the system's operational viability, in line with the course requirements.

1.3 Scope of the Testing Mandate

To ensure methodological rigor and comprehensive validation, a multi-layered testing strategy was devised and implemented. The scope of this mandate was designed to scrutinize the system at all levels of abstraction, from discrete software units to the integrated system as a whole. The investigation encompassed the following testing paradigms:

Static Testing: Critical analysis of project artifacts, including the SRS and source code, without dynamic execution, to identify defects, ambiguities, and deviations from coding standards at the earliest possible stage.

Unit Testing: Rigorous validation of discrete software units (e.g., functions, classes) within the frontend and backend codebases to verify their logical and functional correctness in isolation.

Integration Testing: Examination of the interoperability between system components, with a primary focus on the data exchange and control flow across the interface between the React frontend and the Node.js/Express backend API.

Functional Testing: Application of black-box testing methodologies to confirm that the system's observable features conform to the behaviors specified in the SRS, including use case, entity management, and state transition scenarios.

API Testing: Direct interrogation of the backend API endpoints to validate their adherence to the specified contract, including request handling, data processing, and response generation.

Non-Functional Testing: Assessment of critical system quality attributes beyond core functionality, including:

- **Performance Testing:** Empirical measurement of the system’s responsiveness, throughput, and stability under simulated load conditions.
- **Security Testing:** Proactive identification of potential security vulnerabilities to ensure data confidentiality, integrity, and availability.
- **Usability Testing:** Evaluation of the human-computer interaction (HCI) aspects of the user interface to gauge its effectiveness, efficiency, and user satisfaction.
- **Browser Compatibility Testing:** Verification of consistent rendering and functionality across a defined set of contemporary web browsers.

Regression Testing: Systematic re-validation of existing functionality to ensure that recent code modifications or defect resolutions have not introduced unintended side effects.

Installation and Deployment Testing: Confirmation that the application can be reliably built, containerized using Docker, and deployed to its target operational environment.

1.4 System Architecture Overview

The Library Management System is architected as a full-stack web application employing the MERN technology stack, a widely adopted paradigm for robust and scalable web service development.

- **MongoDB:** A NoSQL, document-oriented database serving as the persistence layer for all application data.
- **Express.js:** A minimalist backend framework for Node.js, providing the foundation for the RESTful API that encapsulates the system’s business logic.
- **React.js:** A declarative JavaScript library for building the dynamic, single-page application (SPA) that constitutes the client-side user interface.
- **Node.js:** A server-side JavaScript runtime environment that powers the application’s backend services.

Figure 1.1 provides a high-level schematic of the system’s architecture, illustrating the fundamental interactions between the client, server, and database tiers.

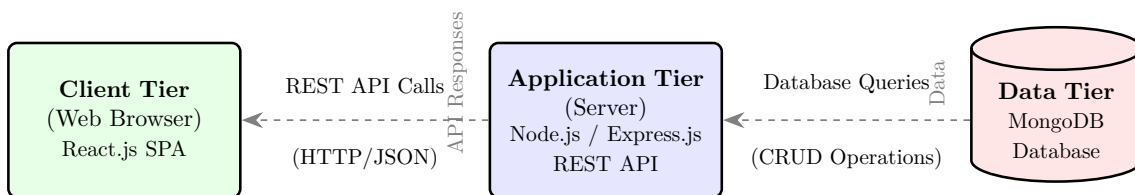


Figure 1.1: High-Level System Architecture of the LMS.